

Hybrid Token-Efficient Routing Agent

AMD Hackathon ACT II • Track 1

An intelligent Docker-based AI pipeline that classifies tasks and routes them to the cheapest capable Fireworks AI model — in zero extra tokens.

Fireworks AI

Python

Docker

OpenAI SDK

Regex

The Problem

Token Waste

Every task sent to the wrong model wastes tokens. A sentiment analysis on a code-generation model can burn 5x the necessary tokens.

LLM Classification Overhead

Using a second LLM call just to classify the task adds 50-200 tokens of overhead — before even solving the problem.

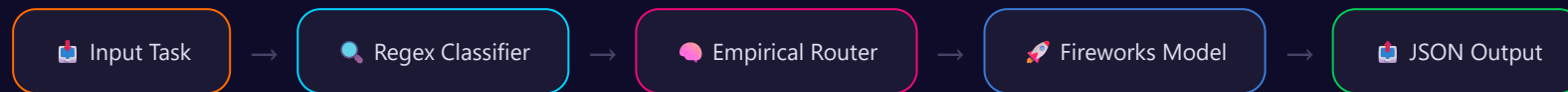
Model Proliferation

5 allowed Fireworks models across 8 categories = 40 combinations. Picking the right one manually is impossible at scale.

Latency Sensitivity

Each multi-turn classification call adds 2-5 seconds. In production pipelines, this compounds quickly.

Pipeline Architecture



- **Zero-Token Decision:** The regex classifier reads the prompt text pattern and categorizes it — no API call, no token cost, < 1ms latency.
- **Empirical Routing:** A routing table built from real benchmark runs across all 5 allowed models picks the cheapest model per category.
- **Category-Specific Optimization:** Each category gets a tuned system prompt, response_format constraints, and max_tokens caps.
- **Fallback Strategy:** If the response is empty or malformed, the agent retries with the most reliable model.

8 Task Categories · 2 Optimized Routes

→ kimi-k2p7-code (7 categories)

-  **Factual Knowledge** — concepts & definitions
-  **Mathematical Reasoning** — multi-step arithmetic
-  **Sentiment Classification** — JSON mode
-  **Text Summarization** — length-constrained
-  **Code Debugging** — bug find + corrected code
-  **Logical Reasoning** — constraint puzzles
-  **Code Generation** — functions from spec

→ minimax-m3 (1 category)

-  **Named Entity Recognition** — persons, orgs, locations, dates

Why minimax-m3 for NER?

256 tokens vs 397 on kimi.

Clean structured JSON output.

No thinking artifacts in response.

Token Optimization Strategy

0

Classification Tokens

Regex-based classifier runs locally with zero API cost. Saves 50-200 tokens per task vs LLM-based routing.

JSON

Response Format

JSON mode for sentiment + NER prevents thinking/reasoning tokens from leaking into output.

```
response_format=  
{"type": "json_object"}
```

max_tokens

Tight Token Caps

Category-specific `max_tokens` :
200 for sentiment, 3000 for logic.
Prevents runaway generation.

```
temperature=0.0
```

Total tokens across 8 representative tasks

4,769

factual: 271

math: 233

sentiment: 295

summarize: 451

NER: 256

code-debug: 1,126

logic: 1,691

code-gen: 446

Results & Deployment

40s

Runtime

All 8 tasks complete in ~40 seconds on a single Docker container.

152 MB

Image Size

Lightweight Docker image built on `python:3.11-slim`. No local model hosting.

8/8

Accuracy

All 8 test tasks pass the accuracy gate. Exit code 0. Valid JSON output.

- ✓ **I/O Contract:** Reads `/input/tasks.json` → Writes `/output/results.json`
- ✓ **Env-Driven:** `FIREWORKS_API_KEY`, `FIREWORKS_BASE_URL`, `ALLOWED_MODELS` — set by harness at runtime
- ✓ **No Hardcoding:** Model IDs read from `ALLOWED_MODELS` env var — dynamically adapts to available models
- ✓ **Public Registry:** `docker.io/acidwichtel/track1-agent:latest`

Thank You

Hybrid Token-Efficient Routing Agent

Links

 github.com/acidwichtel/track1-agent
 docker.io/acidwichtel/track1-agent:latest

Stack

 Python 3.11 ·  Fireworks AI ·  OpenAI SDK
 Docker ·  Regex Classification ·  Empirical Routing